Tredence Analytics

JOB DESCRIPTION – AI Engineering Intern

Python • FastAPI • OpenAI APIs • RAG • Vector DBs • SQL

Location: Bengaluru
Duration: 6 months to 1 year (extendable / convertible), PPO will be given based on performance.

Stipend : 30k Per Month/-

About the Role

We are looking for an AI Engineering Intern who can work with our engineering team to build LLM-powered features, retrieval systems, and agent-style workflows. This role is ideal for someone strong in Python who wants hands-on exposure to modern AI engineering and backend systems.

Responsibilities

- Write clean, maintainable **Python** code for data processing, integrations, and automation.
- Build and maintain API endpoints using **FastAPI**, including async workflows where needed.
- Use **OpenAI and other LLM APIs** for generation, summarization, and extraction tasks.
- Build basic **RAG pipelines**: chunking, embedding, vector indexing, and retrieval.
- Work with vector DBs such as **FAISS, Chroma, or Milvus** for embedding search.
- Write SQL queries and work with relational databases like PostgreSQL/MySQL.
- Implement simple caching (in-memory, Redis) to optimize repeated requests.
- Use Python concepts such as **asyncio, threading, and multiprocessing** where applicable.
- Participate in performance tuning by profiling Python code and optimizing bottlenecks.
- Contribute to documentation, testing, logging, and debugging of AI pipelines.
- Support small agent workflow prototypes and internal tool development.

Required Skills

- Strong command of **Python** and understanding of core libraries.

Tredence Analytics

- Experience developing APIs with **FastAPI**.
- Familiarity with **async programming**, threading, and multiprocessing.
- Understanding of **LLM API usage** (OpenAI or similar).
- Knowledge of **RAG fundamentals** and experience with vector databases.
- Ability to write clear and efficient **SQL** for querying and schema design.
- Basic understanding of caching strategies (in-memory, Redis).
- Solid **problem-solving and DSA** fundamentals.
- Awareness of clean code practices and performance-minded programming.

Bonus / Nice-to-Have

- Exposure to **PyTorch** or other ML frameworks.
- Understanding of fine-tuning (LoRA or similar).
- Familiarity with **LangChain, LangGraph, or Google ADK**.
- Exposure to RLHF concepts.
- Experience with Kafka, RabbitMQ, Celery, or Airflow (even academic).
- Basic Docker or cloud familiarity.

Key Criteria

- **Builder Mindset**: You enjoy creating things end-to-end, iterating quickly, and figuring out practical solutions.
- **Code Quality & Performance Awareness**: You care about writing efficient, reliable, and clean code.
- **Problem-Solving Ability**: Comfortable breaking down problems and applying DSA concepts when needed.
- **Learning Orientation**: Open to feedback, curious, and motivated to pick up new tools and workflows.

What You'll Gain

- Experience contributing to real AI systems across LLMs, RAG, and backend engineering.
- Hands-on exposure to modern AI infra (FastAPI, vector DBs, caching, async workflows).

Tredence Analytics

- Mentorship from engineers working on production-grade AI systems.
- Opportunity for PPO or extension based on performance.

Case study – AI Engineer

Problem Title: The Self-Pruning Neural Network

Preamble

In the real world, deploying large neural networks is often constrained by memory and computational budgets. A common technique to make models smaller and faster is "pruning," where you remove less important weights or neurons after training.

This challenge takes that idea a step further. Your task is to design and implement a neural network that **learns to prune itself *during* the training process**. Instead of a post-training step, the network will have a built-in mechanism to identify and dynamically remove its own weakest connections, adapting its architecture on the fly. This requires a clever application of regularization and a custom training loop.

The Core Problem

You will build a standard feed-forward neural network for an image classification task (e.g., on CIFAR-10). However, you must augment this network with a mechanism that allows it to learn which of its own weights are unnecessary.

The core idea is to associate each weight in the network with a learnable "gate" parameter. This gate, a single scalar value between 0 and 1, will multiply the weight's output. If a gate's value becomes 0, it effectively "prunes" or removes the corresponding weight from the network.

Your goal is to formulate a loss function and training procedure that encourages most of these gates to become exactly zero, leaving only a "sparse" network of the most important connections.

Detailed Task Specification

Part 1: The "Prunable" Linear Layer

Tredence Analytics

You cannot use the standard torch.nn.Linear layer directly. You must create your own custom "prunable" version.

- **Class Signature**: PrunableLinear(in_features, out_features)
- **Implementation Details**:
  1. This layer will have the standard weight and bias parameters.
  2. Crucially, it will also have a second parameter tensor, let's call it gate_scores, with the exact same shape as the weight tensor. This tensor should also be registered as a model parameter so it gets updated by the optimizer.
  3. The forward pass of this layer is where the logic happens:
     - Apply a transformation to the gate_scores to turn them into gates between 0 and 1. A simple and effective choice is the **Sigmoid function**. Let's call the result gates.
     - Calculate the "pruned weights" by performing an element-wise multiplication: pruned_weights = self.weight * gates.
     - Perform the standard linear layer operation using these pruned_weights and the bias.
  - **Challenge**: You must implement this from scratch, ensuring that gradients flow correctly through both the weight and gate_scores parameters.

Part 2: The Sparsity Regularization Loss

If you only train with the standard classification loss (e.g., Cross-Entropy), the network will have no reason to set any gates to zero. You must add a custom regularization term to the loss function that penalizes the network for having too many active gates.

- **Loss Formulation**: Total Loss = ClassificationLoss + λ * SparsityLoss
- **Sparsity Loss Calculation**:
  1. After each forward pass, you need to calculate a penalty based on the state of all the gates in your network.
  2. A good choice for SparsityLoss is the **L1 norm** of all the gate values. The L1 norm (sum of absolute values) is known to encourage sparsity (driving values to exactly zero). Since our gates are always positive (after the sigmoid), this is simply the sum of all gate values across all PrunableLinear layers.
  3. λ (lambda) is a hyperparameter that controls the trade-off. A higher λ will result in a more heavily pruned (sparser) network, at the potential cost of accuracy.

Part 3: Training and Evaluation

You will write a script to train your self-pruning network on the CIFAR-10 dataset.

Tredence Analytics

- **Training Loop**: Your training loop will compute the Total Loss as described above and use a standard optimizer (e.g., Adam) to update all parameters, including the gate_scores.
- **Evaluation**:
    1. After training, you need to report on the final state of the network.
    2. **Calculate and report the "sparsity level"**: This is the percentage of weights whose corresponding gate value is below a small threshold (e.g., 1e-2). A high sparsity level means your method was successful.
    3. **Report the final test accuracy** of the pruned model.
    4. **Compare results** for at least three different values of λ (e.g., a low, medium, and high value) to show how it affects the sparsity-vs-accuracy trade-off.

Input

- The CIFAR-10 dataset, accessible via torchvision.datasets.

Output

1. **Source Code**: A single, well-commented Python script that contains:
    - The PrunableLinear module implementation.
    - The definition of your neural network using these prunable layers.
    - The complete training and evaluation loop.
2. **A Short Report (in Markdown)**:
    - A brief explanation of why an L1 penalty on the sigmoid gates encourages sparsity.
    - A table summarizing your results for different λ values. The table should have columns for: Lambda, Test Accuracy, and Sparsity Level (%).
    - A matplotlib plot showing the distribution of the final gate values for your best model. A successful result will show a large spike at 0 and another cluster of values away from 0.

Evaluation Criteria

1. **Correctness of the PrunableLinear Layer**: Does it correctly implement the gated weight mechanism? Do gradients flow properly?
2. **Successful Implementation of the Training Loop**: Does your code correctly calculate and apply the custom sparsity loss?
3. **Quality of Results and Analysis**: Do your results show that the network is successfully pruning itself? Is your analysis of the λ trade-off clear and logical?
4. **Code Quality**: Is the code clean, readable, and easy to run?

Tredence Analytics

Instructions :

Dear Student,

Tredence Studio's AI Agents Engineering team is inviting applications for the **AI Engineering Internship – 2025 Cohort**.
This internship offers a rare opportunity to work on **frontier engineering and research** in the fast-growing domain of **AI Agent Engineering**.

If you are genuinely passionate about building the next generation of intelligent systems, this is for you.

This is a **hands-on**, engineering-heavy internship. Expect to write real production Python/C++ code, train models, design agent workflows, and experiment with RL-based methods.

Who Should Apply — Self-Selection Criteria

Please apply **only if you meet most of the criteria below**:

Required

- 2–3+ years of **consistent, hands-on programming experience**(Python, C++, Rust or equivalent)
- Strong fundamentals in:
  - Data Structures & Algorithms
  - Probability, Linear Algebra, Calculus
  - Machine Learning theory
- Experience with:
  - Python + PyTorch
  - Deep Learning models
  - **Model fine-tuning (any method)**
  - Basic understanding of **Reinforcement Learning concepts**
  - FastAPI or any backend framework
  - LLMs (fine-tuning, prompting, embeddings, vector DBs)

Bonus (Not mandatory but strong signals)

- Any research paper, blog, or open-source contribution

Tredence Analytics

- Built your own ML project, agent, chatbot, or model training pipeline
- Experience with CUDA, GPU training, or optimization
- Competitive programming background (CodeChef/Codeforces)
- Participation in hackathons or Kaggle competitions

Next Step — Case Study Submission

There is a Coding Assignment document attached with this email. Those who wish to proceed may respond with:

1. **Your resume (PDF)**
2. **GitHub / Portfolio / Project links**
3. **Your completed Coding Assignment** (attached in this email) in a Github Repo (Share the Github Link)

Please complete the case study **independently**. We are evaluating your **problem-solving skills, code quality, clarity of thought, and ability to reason** — not just your final answer.

Important Notes

- Internship Location: India (Bangalore/Hybrid based on team alignment)
- Duration: 6 months to 1 year
- Stipend: As per company standards
- Only shortlisted candidates will be contacted for interviews.

Note : Kindly submit your GitHub URLs without fail

We look forward to reviewing your work and identifying the next generation of exceptional AI Engineers.
If you believe you have the depth, passion, and discipline to work on cutting-edge AI Agent systems, we'd love to see your application.

Best regards,
Tredence Studio — AI Agents Engineering Team